

1 Appendix

1.1 Pseudocodes

Algorithm 1 Main-Background Base Learner with Hyperparameter Optimization

Input: hyperparameter space $\mathcal{H}$, periodicity for performance assesment p , drift detector $\mathcal{DD}$, Loss function $\mathcal{L}$

```

1: function INITIALIZE( )
2:    $C \leftarrow$  get default hyperparameters from  $\mathcal{H}$ 
3:    $C' \leftarrow$  randomly sample hyperparameters from  $\mathcal{H}$ 
4:    $\mathcal{M} \leftarrow$  HoeffdingTree( $C$ ) ;  $\mathcal{B} \leftarrow$  HoeffdingTree( $C'$ )
5:    $\mathcal{L}_{\mathcal{M}} \leftarrow 0$ ;  $\mathcal{L}_{\mathcal{B}} \leftarrow 0$  /* Track accuracy of Main and Background
6:    $\mathcal{I} \leftarrow 0$  /* Amount of instances for periodic assesment
7: end function
8: function TRAIN(instance vector  $\mathbf{x}$ , discrete label  $y$ ) /* One instance at a time
   arriving from the stream
9:    $\mathcal{I} \leftarrow \mathcal{I} + 1$ 
10:  /* Periodically assess performance and search new configurations
11:  if  $\mathcal{I} = p$  then
12:    if  $\mathcal{L}_{\mathcal{B}} > \mathcal{L}_{\mathcal{M}}$  then
13:       $\mathcal{M} \leftarrow \mathcal{B}$ 
14:    end if
15:     $C' \leftarrow$  randomly sample hyperparameters from  $\mathcal{H}$ 
16:     $\mathcal{B} \leftarrow$  HoeffdingTree( $C'$ )
17:     $\mathcal{L}_{\mathcal{M}} \leftarrow 0$ ;  $\mathcal{L}_{\mathcal{B}} \leftarrow 0$ ;  $\mathcal{I} \leftarrow 0$ 
18:  end if
19:  Update  $\mathcal{DD}$  with  $\mathbf{1}\{\text{Predict}(\mathbf{x}) \neq y\}$ 
20:  if  $\mathcal{DD}$  triggers a change then
21:     $C' \leftarrow$  randomly sample hyperparameters from  $\mathcal{H}$ 
22:     $\mathcal{M} \leftarrow$  HoeffdingTree( $C$ ) ;  $\mathcal{B} \leftarrow$  HoeffdingTree( $C'$ )
23:     $\mathcal{L}_{\mathcal{M}} \leftarrow 0$ ;  $\mathcal{L}_{\mathcal{B}} \leftarrow 0$ ;  $\mathcal{I} \leftarrow 0$ 
24:  end if
25:  Update  $\mathcal{L}_{\mathcal{M}}$  and  $\mathcal{L}_{\mathcal{B}}$ 
26:   $\mathcal{M}.\text{Train}(\mathbf{x}, y)$ 
27:   $\mathcal{B}.\text{Train}(\mathbf{x}, y)$ 
28: end function
29: function PREDICT(instance vector  $\mathbf{x}$ )
30:   if  $\mathcal{L}_{\mathcal{M}} > \mathcal{L}_{\mathcal{B}}$  then
31:     return  $\mathcal{M}.\text{Predict}(\mathbf{x})$ 
32:   else
33:     return  $\mathcal{B}.\text{Predict}(\mathbf{x})$ 
34:   end if
35: end function

```

1.2 *SORE* Ablation

Table 1: Accuracy of *SORE* compared to SRP, *SORE* without regularization and with Rejection-based regularization

Optimization	✓	✗	✓	✓
Regularization	Subbagging	✗	✗	Rejection
Dataset	<i>SORE</i>	SRP	<i>SORE</i> _{wo/reg}	<i>SORE</i> _{rej.}
LED _a	73.929 ± 0.011	73.891 ± 0.014	73.883 ± 0.009	73.677 ± 0.025
LED _g	<u>72.951</u> ± 0.012	73.008 ± 0.022	72.885 ± 0.018	72.545 ± 0.040
SEA _a	89.208 ± 0.040	88.140 ± 0.200	89.659 ± 0.012	<u>89.421</u> ± 0.018
SEA _g	88.564 ± 0.033	86.970 ± 0.282	88.941 ± 0.008	<u>88.586</u> ± 0.032
AGR _a	93.021 ± 0.184	92.654 ± 0.127	<u>92.812</u> ± 0.268	92.339 ± 0.450
AGR _g	<u>89.357</u> ± 0.308	89.312 ± 0.210	90.339 ± 0.159	88.349 ± 0.300
RBF _m	<u>86.784</u> ± 0.055	86.062 ± 0.020	85.548 ± 0.052	86.853 ± 0.055
RBF _f	79.170 ± 0.088	76.368 ± 0.025	77.543 ± 0.047	<u>78.102</u> ± 0.311
HYPER	86.412 ± 0.122	<u>85.568</u> ± 0.078	85.320 ± 0.124	85.059 ± 0.199
CaDrift _a	88.484 ± 0.050	<u>87.917</u> ± 0.057	87.259 ± 0.032	87.342 ± 0.066
CaDrift _g	83.840 ± 0.090	<u>82.376</u> ± 0.123	82.218 ± 0.187	80.833 ± 0.220
CaDrift _i	94.001 ± 0.035	93.108 ± 0.062	<u>93.385</u> ± 0.111	93.108 ± 0.286
Outdoor	<u>76.582</u> ± 0.333	68.517 ± 0.515	73.688 ± 0.956	77.198 ± 0.453
Elec	92.993 ± 0.047	89.750 ± 0.102	91.241 ± 0.090	<u>92.519</u> ± 0.065
Poker	<u>94.259</u> ± 0.215	89.766 ± 0.070	93.036 ± 0.168	94.523 ± 0.435
Rialto	82.190 ± 0.282	80.126 ± 0.094	82.300 ± 0.274	<u>82.199</u> ± 0.407
Sensor	<u>93.988</u> ± 0.420	94.139 ± 0.070	93.097 ± 0.423	93.908 ± 0.099
CovType	96.089 ± 0.025	95.344 ± 0.020	95.462 ± 0.021	<u>95.636</u> ± 0.035
Gas	95.753 ± 0.109	93.457 ± 0.121	<u>95.134</u> ± 0.125	94.948 ± 0.097
NOAA	79.004 ± 0.076	<u>79.408</u> ± 0.076	79.737 ± 0.051	78.323 ± 0.100
INSECTS _a	<u>76.635</u> ± 0.143	76.266 ± 0.056	77.046 ± 0.108	75.874 ± 0.270
INSECTS _i	65.751 ± 0.288	<u>65.820</u> ± 0.088	66.397 ± 0.070	64.026 ± 0.347
INSECTS _g	<u>79.610</u> ± 0.104	79.370 ± 0.103	79.683 ± 0.128	78.995 ± 0.186
LADPU	69.669 ± 0.109	71.873 ± 0.133	69.595 ± 0.284	<u>71.200</u> ± 0.555
Asfalt	88.514 ± 0.073	88.981 ± 0.134	<u>88.967</u> ± 0.096	88.663 ± 0.000
Mean Acc.	84.670	83.528	<u>84.207</u>	84.169
Mean Acc. Synth.	85.477	84.615	<u>84.983</u>	84.685
Mean Acc. Real.	83.926	82.524	83.491	<u>83.693</u>
Mean Rank	1.92	2.80	<u>2.36</u>	2.92
Mean Rank Synth	1.58	2.75	<u>2.50</u>	3.17
Mean Rank Real	2.23	2.85	2.23	<u>2.69</u>

Table 2: F1 Score of SORE compared to SRP, SORE without regularization and with Rejection-based regularization

Optimization	✓	✗	✓	✓
Regularization	Subbagging	✗	✗	Rejection
Dataset	<i>SORE</i>	SRP	<i>SORE_{wo/reg}</i>	<i>SORE_{rej.}</i>
LED _a	74.071 ± 0.032	73.811 ± 0.016	73.888 ± 0.014	73.667 ± 0.025
LED _g	73.084 ± 0.015	<u>72.917</u> ± 0.023	72.851 ± 0.013	72.535 ± 0.039
SEA _a	88.379 ± 0.044	87.215 ± 0.218	88.870 ± 0.013	<u>88.611</u> ± 0.020
SEA _g	87.678 ± 0.036	85.933 ± 0.308	88.087 ± 0.009	<u>87.700</u> ± 0.035
AGR _a	92.866 ± 0.188	92.499 ± 0.127	<u>92.644</u> ± 0.275	92.232 ± 0.445
AGR _g	<u>89.119</u> ± 0.316	89.089 ± 0.214	90.118 ± 0.163	88.185 ± 0.295
RBF _m	<u>86.737</u> ± 0.054	86.090 ± 0.018	85.505 ± 0.051	86.854 ± 0.053
RBF _f	78.888 ± 0.086	76.378 ± 0.025	77.309 ± 0.042	<u>77.802</u> ± 0.348
HYPER	86.414 ± 0.122	<u>85.594</u> ± 0.077	85.323 ± 0.124	85.066 ± 0.198
CaDrift _a	87.822 ± 0.058	<u>87.207</u> ± 0.066	86.516 ± 0.039	86.606 ± 0.075
CaDrift _g	80.624 ± 0.201	<u>78.919</u> ± 0.156	78.669 ± 0.265	77.317 ± 0.229
CaDrift _i	86.111 ± 0.720	<u>85.232</u> ± 1.276	85.107 ± 1.308	85.215 ± 1.723
Outdoor	78.342 ± 0.334	70.999 ± 0.486	75.476 ± 0.882	79.157 ± 0.364
Elec	92.818 ± 0.048	89.480 ± 0.105	91.016 ± 0.093	<u>92.328</u> ± 0.067
Poker	57.016 ± 0.330	49.506 ± 0.190	52.146 ± 0.255	<u>56.553</u> ± 0.541
Rialto	<u>82.359</u> ± 0.266	80.697 ± 0.091	82.531 ± 0.248	82.353 ± 0.364
Sensor	89.393 ± 1.595	87.481 ± 0.098	<u>89.853</u> ± 0.839	90.951 ± 0.093
CovType	92.540 ± 0.066	91.666 ± 0.053	91.364 ± 0.030	<u>91.903</u> ± 0.073
Gas	95.719 ± 0.101	93.594 ± 0.110	95.104 ± 0.121	95.048 ± 0.087
NOAA	74.580 ± 0.096	<u>74.683</u> ± 0.101	75.158 ± 0.065	73.785 ± 0.113
INSECTS _a	<u>76.571</u> ± 0.155	76.232 ± 0.055	77.003 ± 0.118	75.781 ± 0.261
INSECTS _i	65.654 ± 0.272	<u>65.802</u> ± 0.087	66.288 ± 0.066	63.964 ± 0.304
INSECTS _g	<u>79.508</u> ± 0.105	79.260 ± 0.105	79.600 ± 0.132	78.896 ± 0.183
LADPU	69.729 ± 0.114	72.446 ± 0.127	69.753 ± 0.289	<u>71.355</u> ± 0.584
Asfalt	<u>80.253</u> ± 0.069	79.593 ± 0.407	80.379 ± 0.167	80.214 ± 0.000
Mean Acc.	81.851	80.493	81.222	<u>81.363</u>
Mean Acc. Synth.	84.316	83.407	<u>83.741</u>	83.482
Mean Acc. Real.	79.576	77.803	78.898	<u>79.407</u>
Mean Rank	1.80	3.04	<u>2.28</u>	2.88
Mean Rank Synth	1.50	2.83	<u>2.58</u>	3.08
Mean Rank Real	<u>2.08</u>	3.23	2.00	2.69

1.3 Existing Hyperparameter Optimization frameworks

Single-pass Self Parameter Tuning (SSPT) [14] periodically performs Nelder-Mead operators to generate new configurations based on the top 3 performing existing ones. The authors also proposed a convergence analysis based on the performance of the top 3 models, and the optimization process is restarted just upon concept drift detection.



Fig. 1: Wilcoxon post-hoc tests on Accuracy and F1-Score of Components

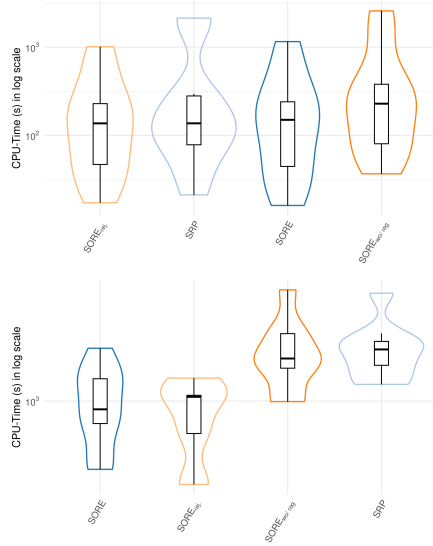


Fig. 2: CPU-Time (in seconds, log-scale) of SORE components on real-world (top) and synthetic (bottom) datasets .

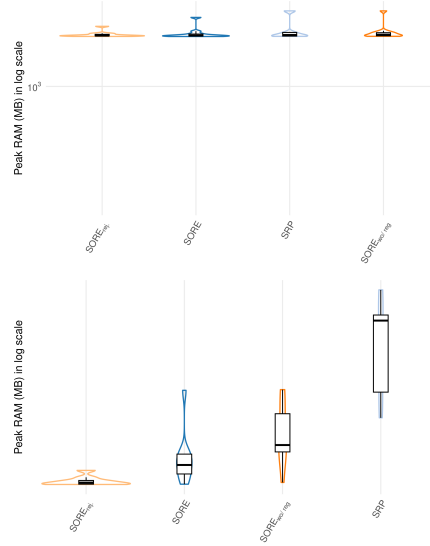


Fig. 3: Peak RAM (in MB, log-scale) of SORE components on real-world (top) and synthetic (bottom) datasets of SORE ablation.

Micro-Evolutionary Single-pass Self Parameter Tuning (MESSPT) [11] uses a micro-differential evolution strategy that evolves candidate solutions by combining existing ones through mutation (adding scaled differences between individuals) and crossover, followed by selection of the best solutions.

Bayesian Stream Tuner (BST) [15] instantiates a surrogate online Bayesian linear regressor, which is periodically trained with descriptive information of the instances features, such as mean, median, kurtosis, etc... combined with the hyperparameter configuration values, and the model performance in the last p instances serves as a target. BST replaces half of worst-performing algorithms with new models with configurations sampled with the highest acquisition function values.

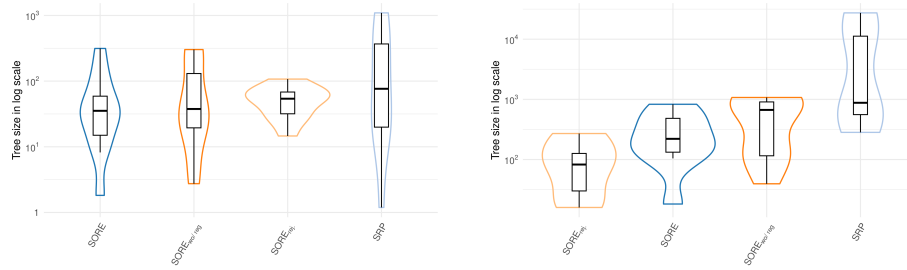


Fig. 4: Tree size (number of nodes, log scale) of SORE components on real-world (left) and synthetic (right) datasets of SORE ablation.



Fig. 5: Wilcoxon post-hoc tests on Accuracy and F1-Score of Components

1.4 Results on existing Hyperparameter Optimization frameworks

This Section shows results for Existing Hyperparameter Optimization frameworks, with the same experimental protocol from the paper and default configurations for the frameworks for a single Hoeffding Tree [8].

Table 3: Kappa of ensembles
Bold values indicate the best results per category
Underlined values indicate second best results per category
- values indicate out of memory and are treated as the worst results

Dataset	LEVBAG	ARF	SRP	ARE	ARTE	SGBT	SORE
LED _a	70.754 ± 0.026	70.946 ± 0.012	70.990 ± 0.015	70.529 ± 0.663	70.841 ± 0.162	71.067 ± 0.010	<u>71.031</u> ± 0.012
LED _g	69.930 ± 0.015	69.896 ± 0.010	<u>70.010</u> ± 0.025	69.572 ± 0.455	69.759 ± 0.151	70.179 ± 0.010	69.945 ± 0.014
SEA _a	73.119 ± 0.149	<u>77.440</u> ± 0.018	74.104 ± 0.459	77.110 ± 0.007	77.476 ± 0.004	76.435 ± 0.049	76.569 ± 0.088
SEA _g	74.514 ± 0.022	<u>75.703</u> ± 0.013	71.455 ± 0.655	75.572 ± 0.013	75.844 ± 0.009	75.054 ± 0.058	75.165 ± 0.073
AGR _a	69.037 ± 0.768	72.901 ± 0.412	84.997 ± 0.255	72.082 ± 2.582	61.356 ± 0.136	90.285 ± 0.013	<u>85.730</u> ± 0.377
AGR _g	65.627 ± 0.098	63.352 ± 0.379	78.176 ± 0.428	63.378 ± 2.436	54.481 ± 0.073	84.186 ± 0.028	<u>78.237</u> ± 0.632
RBF _m	80.624 ± 0.038	83.286 ± 0.027	81.817 ± 0.026	83.801 ± 0.375	<u>83.644</u> ± 0.339	83.288 ± 0.022	82.761 ± 0.072
RBF _f	47.022 ± 0.052	69.949 ± 0.031	68.821 ± 0.032	68.037 ± 0.888	<u>71.707</u> ± 1.673	67.438 ± 0.028	72.620 ± 0.118
HYPER	75.178 ± 0.046	71.552 ± 0.042	71.136 ± 0.155	<u>75.088</u> ± 0.764	73.160 ± 0.646	73.902 ± 0.073	72.824 ± 0.244
CaDrift _a	81.723 ± 0.075	85.664 ± 0.028	<u>86.559</u> ± 0.064	85.991 ± 0.797	85.064 ± 0.730	–	87.194 ± 0.057
CaDrift _g	75.869 ± 0.077	77.955 ± 0.044	79.734 ± 0.144	<u>79.853</u> ± 1.361	78.210 ± 0.917	–	81.442 ± 0.104
CaDrift _i	88.509 ± 0.425	89.742 ± 0.085	<u>92.076</u> ± 0.071	90.730 ± 0.317	90.755 ± 0.506	–	93.102 ± 0.041
Outdoor	56.418 ± 0.152	59.738 ± 0.160	67.710 ± 0.528	68.587 ± 3.531	<u>68.736</u> ± 3.935	–	75.982 ± 0.342
Elec	77.111 ± 0.201	78.606 ± 0.140	78.913 ± 0.209	<u>82.148</u> ± 0.459	80.676 ± 1.218	75.919 ± 0.175	85.627 ± 0.097
Poker	91.676 ± 0.312	79.068 ± 0.057	81.322 ± 0.130	86.223 ± 3.988	69.817 ± 7.674	80.969 ± 0.055	<u>89.667</u> ± 0.390
Rialto	56.138 ± 0.414	70.674 ± 0.113	77.918 ± 0.105	69.696 ± 4.955	78.630 ± 1.713	84.308 ± 0.080	<u>80.211</u> ± 0.313
Sensor	89.513 ± 0.015	89.208 ± 0.017	94.012 ± 0.072	93.256 ± 0.379	92.720 ± 0.006	–	<u>93.858</u> ± 0.429
CovType	89.155 ± 0.052	91.978 ± 0.021	92.508 ± 0.032	<u>92.557</u> ± 0.138	91.480 ± 1.487	90.732 ± 0.028	93.710 ± 0.040
Gas	76.500 ± 0.596	89.730 ± 0.209	92.062 ± 0.147	90.096 ± 1.610	<u>94.501</u> ± 0.679	88.068 ± 0.330	94.847 ± 0.132
NOAA	44.675 ± 1.214	48.346 ± 0.242	47.461 ± 0.210	49.114 ± 0.173	48.276 ± 0.420	46.212 ± 0.485	48.604 ± 0.194
INSECTS _a	62.671 ± 0.124	70.095 ± 0.100	71.520 ± 0.067	68.809 ± 0.359	72.483 ± 0.030	70.836 ± 0.061	<u>71.962</u> ± 0.172
INSECTS _i	55.105 ± 0.107	58.442 ± 0.097	58.985 ± 0.106	59.101 ± 0.143	<u>59.503</u> ± 0.067	59.845 ± 0.086	58.901 ± 0.346
INSECTS _g	65.725 ± 0.272	73.689 ± 0.065	75.244 ± 0.124	72.529 ± 0.293	<u>75.442</u> ± 0.094	74.490 ± 0.107	75.532 ± 0.125
LADPU	57.581 ± 0.250	63.512 ± 0.136	<u>68.748</u> ± 0.148	65.410 ± 0.636	67.316 ± 1.219	72.909 ± 0.327	66.299 ± 0.121
Asfalt	79.110 ± 0.099	80.937 ± 0.273	<u>81.661</u> ± 0.221	80.022 ± 0.616	82.636 ± 0.252	76.389 ± 0.284	80.704 ± 0.126
Mean Acc.	70.931	74.497	<u>76.718</u>	75.572	74.980	75.626	78.501
Mean Acc. Synth.	72.659	75.699	<u>77.490</u>	75.978	74.358	76.870	78.885
Mean Acc. Real.	69.337	73.387	<u>76.005</u>	75.196	75.555	74.607	78.147
Mean Rank	5.84	4.56	3.60	3.84	<u>3.44</u>	4.36	2.36
Mean Rank Synth	5.50	4.17	4.00	<u>3.92</u>	<u>3.92</u>	<u>3.92</u>	2.58
Mean Rank Real	6.15	4.92	3.23	3.77	<u>3.00</u>	4.77	2.15

2 Datasets configurations

Table 4 shows the datasets’ characteristics. We generate datasets of 1 million instances with concept drift and 500 thousand for CaDrift. We follow a protocol by [7] which extends existing works [9], [3] that uses the synthetic dataset suites from the MOA [5] framework, to contain a higher number of drifts. [7] generate 6 different types of datasets with different concept order for AGR, SEA, and LED, while 3 for RBF and HYPER. We match the number of datasets to the work of [9], aiming to reduce that a good result in a single dataset inflates other results.

The simulation of abrupt and gradual concept drifts in synthetic datasets consists of the interleaved generation of instances from a concept C_1 or C_2 . Then, a transition period is defined, which determines the probability of generating an instance from concept C_1 or C_2 .

We follow the approach of [4] that defines a sigmoid probabilistic function for determining the probability that an instance will be generated given the concept C_1 , otherwise C_2 . Given an instance I generated at time t_n , with the drift beginning at time t_d and lasting for w instances, the probability that this instance belongs to concept C_1 , or $P[I \in C_1]$, is defined in Equation 1.

Dataset	BST	RS	SSPT	MESSPT
AGR _a	90.240 ± 0.810	89.960 ± 0.950	86.970 ± 1.000	86.590 ± 1.200
AGR _g	83.200 ± 0.800	82.910 ± 1.030	81.840 ± 1.060	81.620 ± 1.180
LED _a	72.930 ± 0.160	<u>72.920</u> ± 0.170	67.250 ± 1.400	70.400 ± 0.760
LED _g	71.800 ± 0.160	<u>71.600</u> ± 0.310	67.340 ± 0.900	69.190 ± 0.580
SEA _a	88.160 ± 0.070	87.920 ± 0.090	87.820 ± 0.210	<u>88.000</u> ± 0.080
SEA _g	87.540 ± 0.060	<u>87.350</u> ± 0.130	86.700 ± 0.290	86.690 ± 0.160
RBF _m	72.550 ± 0.350	72.060 ± 0.520	68.700 ± 0.650	68.270 ± 1.520
RBF _f	<u>57.830</u> ± 0.300	57.900 ± 0.250	51.190 ± 1.080	54.790 ± 0.470
HYPER	86.870 ± 0.120	<u>86.590</u> ± 0.130	84.980 ± 1.240	85.820 ± 0.220
Elec	84.730 ± 0.420	<u>84.680</u> ± 0.460	82.100 ± 0.920	82.260 ± 0.760
Rialto	<u>53.590</u> ± 1.200	54.080 ± 0.800	47.710 ± 2.330	44.860 ± 2.370
CovType	85.130 ± 0.170	<u>84.880</u> ± 0.190	84.070 ± 0.560	81.950 ± 0.420
Nomao	94.860 ± 0.210	<u>94.780</u> ± 0.140	92.350 ± 0.950	92.870 ± 0.870
Poker	74.570 ± 0.070	74.610 ± 0.080	71.370 ± 0.250	72.200 ± 0.200
NOAA	74.780 ± 0.770	<u>74.150</u> ± 0.800	73.480 ± 1.100	73.190 ± 1.510
INSECTS _a	68.300 ± 1.160	<u>67.600</u> ± 0.560	62.510 ± 2.940	63.480 ± 1.170
INSECTS _i	58.650 ± 0.710	<u>57.770</u> ± 1.230	53.800 ± 2.050	54.080 ± 1.380
INSECTS _g	71.100 ± 0.770	<u>70.300</u> ± 0.680	65.000 ± 2.590	66.470 ± 1.570
LADPU	61.230 ± 0.590	<u>60.880</u> ± 0.500	49.880 ± 13.980	54.080 ± 2.580
Asfalt	<u>85.420</u> ± 0.160	85.490 ± 0.290	77.650 ± 7.850	80.730 ± 3.730
Mean Acc.	76.174	<u>75.922</u>	72.136	72.877
Mean Acc. Synth.	79.013	<u>78.801</u>	75.866	76.819
Mean Acc. Real.	73.851	<u>73.565</u>	69.084	69.652
Mean Rank	1.20	<u>1.85</u>	3.65	3.30
Mean Rank Synth	1.11	<u>2.00</u>	3.56	3.33
Mean Rank Real	1.27	<u>1.73</u>	3.73	3.27

$$P[I \in C_1] = \frac{1}{1 + e^{-\frac{4}{w} \cdot (t_n - t_d)}} \quad (1)$$

Note that $P[I \in C_1] = 1 - P[I \in C_2]$. For the generation of abrupt concept drifts, the value of w must be low, whereas for gradual concept drifts, the value of w must be high. We follow the approach of [7], which defines $w = 50$ for abrupt drifts and $w = 50000$ for gradual drifts. Given the dataset size $|\mathcal{DS}|$ and the number of concept drifts simulated $\mathcal{CD}$, the transition period between 2 concepts at the i^{th} concept drift of the stream starts at instance $\lfloor i \frac{|\mathcal{DS}|}{|\mathcal{CD}|} \rfloor$.

Incremental concept drifts in this work are inherent to the dataset's problem, such as in the synthetic RBF dataset, where the change in the centroids' positions over time simulates intermediate concepts.

Each dataset is explained as follows.

LED [6] This generator produces 24 boolean features, while 17 are irrelevant. Each feature has a 10% chance of being inverted, simulating noise. Concept drifts are simulated by reducing or increasing the number of relevant features. For both

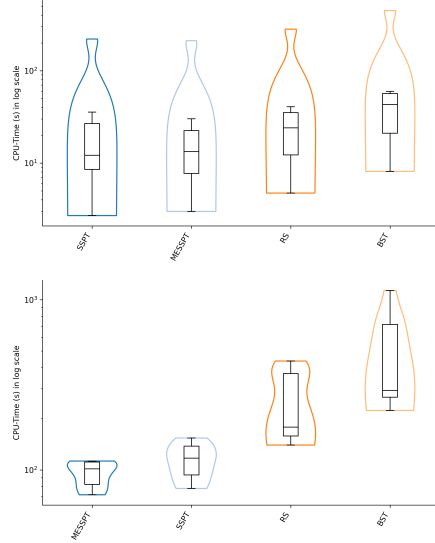


Fig. 6: CPU-Time (in seconds, log-scale) of HPO frameworks on real-world (top) and synthetic (bottom) datasets .

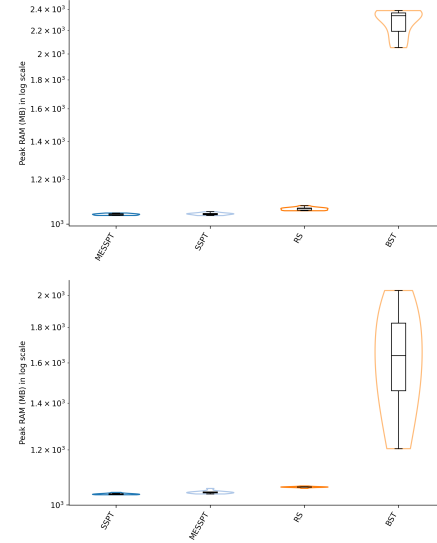


Fig. 7: Peak RAM (in MB, log-scale) of HPO frameworks on real-world (top) and synthetic (bottom) datasets of SORE ablation.

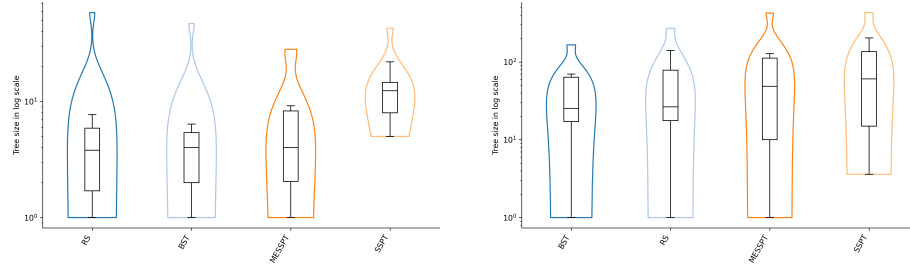


Fig. 8: Tree size (number of nodes, log scale) of SORE components on real-world (left) and synthetic (right) datasets of SORE ablation.

LED_a (with Abrupt drift) and LED_g (Gradual drift) the order of the number of relevant features for each concept is $\{7, 5, 3, 1, 3, 5, 7\}$. Therefore, the stream starts with 7 relevant features, then changes to 5, to 3, and so forth. Therefore LED datasets contain 6 drifts.

SEA [13] This generator produces 3 numerical features (f_1, f_2, f_3) between $[0, 10)$. The dataset contains 4 concepts, being $c_1 = (f_1 + f_2 \leq 8)$, $c_2 = (f_1 + f_2 \leq 9)$, $c_3 = (f_1 + f_2 \leq 7)$, and $c_4 = (f_1 + f_2 \leq 9.5)$. If the generated instance condition is true for c_n , the instance has class value 1, otherwise 0. For both abrupt

Table 4: Dataset characteristics

Dataset	# Samples	# Features	# Classes	Majority (%)	Drift
<i>Synthetic datasets</i>					
LED _a	10 ⁶	24	10	10.28	A
LED _g	10 ⁶	24	10	10.28	G
SEA _a	10 ⁶	3	2	59.91	A
SEA _g	10 ⁶	3	2	59.91	G
AGR _a	10 ⁶	9	2	52.83	A
AGR _g	10 ⁶	9	2	52.83	G
RBF _m	10 ⁶	10	5	30.01	I
RBF _f	10 ⁶	10	5	30.01	I
HYPER	10 ⁶	10	2	50.00	I
CaDrift _a	5 · 10 ⁵	35	11	14.26	A
CaDrift _g	5 · 10 ⁵	35	11	14.26	G
CaDrift _i	5 · 10 ⁵	35	11	14.26	I
<i>Real-world datasets</i>					
Outdoor	4,000	21	40	4.11	U
Elec	45,312	8	2	57.41	U
Poker	829,201	10	10	47.78	U
Rialto	82,250	27	10	10.00	U
Sensor	2,219,803	5	54	2.83	U
CoverType	581,012	54	7	48.75	U
Gas	13,910	128	6	17.18	U
NOAA	18,158	8	2	69.74	U
INSECTS _a	52,848	33	6	16.07	A
INSECTS _i	57,018	33	6	11.56	G
INSECTS _g	24,150	33	6	15.76	I
LADPU	22,950	96	10	10.00	U
Asfalt	8,066	62	5	55.59	U

Drift types: A — Abrupt, G — Gradual, I — Incremental, U — Unknown.

and gradual versions, the order of concepts is $\{c_1, c_2, c_3, c_4, c_3, c_2, c_1\}$. Therefore, the stream starts with concept c_1 , then changes to c_2 , and so forth. This dataset contains 6 drifts.

AGRAWAL [1] This generator has six nominal and three numerical features. Ten distinct functions map two classes. For abrupt datasets, the concept goes from the 1st to the 10th, increasingly, and for gradual data, the concept goes from the 10th to the 1st, decreasingly. Therefore, we simulate 9 concept drifts.

RBF [5] This generator produces ten features and 5 class values. Data are generated based on the radial basis function (RBF). Centroids are generated randomly and mapped with a standard deviation value, a weight, and a class label. In this dataset, incremental drifts are simulated by continuously changing

the centroids' position. The parameters used were 50 centroids at a speed change of 10^{-4} (moderate, RBF_m) and 10^{-3} (fast, RBF_f).

HYPER [10] A hyperplane is a flat, $(n-1)$ dimensional subset of that space that divides it into two disconnected parts. Drifts can be simulated incrementally by changing the decision boundary implied. HYPER was set up with 10 features and a magnitude of change of 10^{-3} .

CaDrift [2] This generator produces synthetic data streams using Structural Causal Models (SCMs), where each feature corresponds to a node in a Directed Acyclic Graph (DAG) representing cause-and-effect relationships between variables and the target. Root nodes are generated from probability distributions such as Normal or Uniform, while inner nodes are computed through mapping functions (e.g., multilayer perceptrons, decision trees, or linear regression) that model dependencies on parent nodes. To introduce time dependency, the generator combines an Exponentially Weighted Moving Average (EWMA) on root nodes with autoregressive noise, creating serial correlation between consecutive samples and making the stream non-iid. Concept drift is simulated by modifying the causal mechanisms of the graph, enabling different drift types such as distributional drift (changing the mapping to the target and affecting $P(y|X)$), covariate drift (changing the distribution of root nodes and affecting $P(X)$), severe drift (inverting class outputs), and local drift (changing the distribution of a single feature). The framework also supports different drift rates, including abrupt, gradual, incremental, and recurrent drifts, as well as interventions that override causal relationships to simulate perturbations, enabling the generation of diverse and controllable evolving data streams.

For real-world data, we refer the interested reader to [12], as they provide a detailed explanation of each dataset.

References

1. Agrawal, R., Imielinski, T., Swami, A.: Database mining: A performance perspective. *IEEE Transactions on Knowledge and Data Engineering* **5**(6), 914–925 (1993). <https://doi.org/10.1109/69.250074>
2. Barboza, E.V.L., Barddal, J.P., Sabourin, R., Cruz, R.M.O.: Cadrift: A time-dependent causal generator of drifting data streams (2026)
3. Bifet, A., Holmes, G., Pfahringer, B.: Leveraging bagging for evolving data streams. In: *Machine Learning and Knowledge Discovery in Databases*. pp. 135–150. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
4. Bifet, A., Holmes, G., Pfahringer, B., Kirkby, R., Gavaldà, R.: New ensemble methods for evolving data streams. In: *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. p. 139–148. KDD '09, Association for Computing Machinery (2009). <https://doi.org/10.1145/1557019.1557041>, <https://doi.org/10.1145/1557019.1557041>

5. Bifet, A., Holmes, G., Pfahringer, B., Kranen, P., Kremer, H., Jansen, T., Seidl, T.: Moa: Massive online analysis, a framework for stream classification and clustering. In: Proceedings of the First Workshop on Applications of Pattern Analysis. Proceedings of Machine Learning Research, vol. 11, pp. 44–50. PMLR (2010), <https://proceedings.mlr.press/v11/bifet10a.html>
6. Breiman, L.: Classification and regression trees. Wadsworth Statistics, Wadsworth, Belmont, CA, (1984)
7. Cano, A., Krawczyk, B.: Kappa updated ensemble for drifting data stream mining. *Machine Learning* **109**(1), 175–218 (2020). <https://doi.org/10.1007/s10994-019-05840-z>, <https://doi.org/10.1007/s10994-019-05840-z>
8. Domingos, P., Hulten, G.: Mining high-speed data streams. In: Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. p. 71–80. KDD '00, Association for Computing Machinery (2000)
9. Gomes, H.M., Bifet, A., Read, J., Barddal, J.P., Enembreck, F., Pfahringer, B., Holmes, G., Abdessalem, T.: Adaptive random forests for evolving data stream classification. *Machine Learning* **106**, 1469–1495 (2017)
10. Hulten, G., Spencer, L., Domingos, P.: Mining time-changing data streams. In: Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 97–106. KDD '01, Association for Computing Machinery, New York, NY, USA (2001)
11. Moya, A.R., Veloso, B., Gama, J., Ventura, S.: Improving hyper-parameter self-tuning for data streams by adapting an evolutionary approach. *Data Mining and Knowledge Discovery* **38**(3), 1289–1315 (2023)
12. Souza, V.M., dos Reis, D.M., Maletzke, A.G., Batista, G.E.: Challenges in benchmarking stream learning algorithms with real-world data. *Data Mining and Knowledge Discovery* **34**, 1805–1858 (2020)
13. Street, W.N., Kim, Y.: A streaming ensemble algorithm (sea) for large-scale classification. In: Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 377–382. KDD '01, Association for Computing Machinery, New York, NY, USA (2001)
14. Veloso, B., Gama, J., Malheiro, B., Vinagre, J.: Hyperparameter self-tuning for data streams. *Information Fusion* **76**, 75–86 (2021)
15. Verma, N., Bifet, A., Pfahringer, B., Bahri, M.: Bayesian stream tuner: Dynamic hyperparameter optimization for real-time data streams. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2. p. 2871–2882. KDD '25, Association for Computing Machinery (2025)